SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Explore Nepal – Travel and Tourism Website

Group C

Student Name	Student ID
Bogati Mamata	2401470
Bhusal Kalpana	2401476
Tamang Sujina	2401537
Bhatta Pabitra	2401467
Mahato Amit Kumar	2401455

Project Type: Full-Stack Web Application

Frontend: HTML, CSS, Bootstrap, JavaScript

Backend: Node.js and Express.js

Database: MySQL

Development Environment: Visual Studio Code

Table of Contents

1. Introduction

2. Overall Description

3. Specific Requirements

4. External Interface Requirements

5. Database Requirements

6. System Models

7. Non-Functional Requirements

8. Security Requirements

9. Project Constraints, Assumptions and Dependencies

10. Future Enhancements

11. Acceptance Criteria

12. Conclusion

Appendix A – Team Information

Appendix B – Suggested Database Tables

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to describe the requirements of the “Explore Nepal – Travel and Tourism Website.” The document explains what the system should do, who will use it, what information it will manage, and how the major components will interact. It provides a common reference for the members of Group C during analysis, design, development, testing, and presentation.

The proposed website is designed to bring useful tourism information into one platform. Users will be able to explore destinations, read travel information, view tour packages, check package details, submit booking requests, and contact the tourism service. Administrators will be able to manage destinations, packages, bookings, users, and messages.

1.2 Scope

Explore Nepal is a web-based travel and tourism information and booking system focused on Nepal. The website will present popular destinations such as Kathmandu, Pokhara, Chitwan, Everest, Mustang, and Lumbini. It will also provide tour packages with information such as package name, destination, duration, description, price, and available services.

The system will contain a public user interface and an administrative management area. The public side will help visitors discover Nepal and submit booking information. The administrative side will allow authorized staff to manage the information displayed on the website and review submitted bookings and contact messages.

The project is intended as an academic full-stack web application demonstrating the integration of frontend technologies, backend APIs, server-side processing, and a relational database.

1.3 Definitions, Acronyms and Abbreviations

- **SRS:** Software Requirements Specification.
- **UI:** User Interface.
- **API:** Application Programming Interface.
- **DB:** Database.
- **Admin:** Authorized person responsible for managing the website.
- **User/Customer:** A visitor or traveler using the website.
- **CRUD:** Create, Read, Update and Delete.
- **DBMS:** Database Management System.

1.4 References

This document follows an IEEE-style SRS organization suitable for an academic software engineering project. The exact formatting may be adapted to the university or instructor's required template.

1.5 Document Overview

Section 1 introduces the project. Section 2 describes the overall system, users, functions, constraints, and assumptions. Section 3 defines functional requirements. Sections 4 and 5 describe interfaces and database requirements. Section 6 describes system models. Sections 7–10 cover quality, security, constraints, and future improvements. The appendices provide team and database information.

2. Overall Description

2.1 Product Perspective

Explore Nepal will operate as a client-server web application. The browser-based frontend will communicate with a Node.js/Express.js backend. The backend will process requests, apply validation and business rules, and communicate with the MySQL database. This architecture separates the presentation layer, application logic, and data storage.

The system may later be connected to external services such as online payment gateways, maps, hotel services, transportation providers, email services, and tourism information APIs.

2.2 Major Product Functions

- Display a responsive home page with tourism highlights.
- Display a list of destinations and detailed destination information.
- Display tour packages with duration, price and description.
- Allow users to search or filter destinations and packages.
- Provide a booking form for selected tour packages.
- Validate and store booking information.
- Provide user registration and login if authentication is implemented.

- Allow users to send questions or messages through the contact form.
- Provide an admin dashboard for managing destinations, packages and bookings.
- Allow administrators to update, add or remove tourism information.
- Store important application data in a MySQL relational database.
- Provide booking status or confirmation information.

2.3 User Classes and Characteristics

- **Tourist/Customer:** General website visitor who searches destinations, reads information, views packages and submits bookings.
- **Registered User:** A customer with an account who can manage personal information and view relevant booking information.
- **Administrator:** Authorized user who manages destinations, packages, users, bookings and messages.
- **Tour Guide/Service Provider:** An external stakeholder who may provide guiding or tourism services.
- **Development Team:** Group C members responsible for analysis, design, implementation, testing and maintenance.

2.4 Operating Environment

- Client devices: desktop computers, laptops, tablets and smartphones.
- Web browsers: modern versions of Chrome, Edge, Firefox or Safari.
- Development environment: Visual Studio Code.
- Server runtime: Node.js.
- Backend framework: Express.js.
- Database server: MySQL.
- Frontend technologies: HTML5, CSS3, Bootstrap and JavaScript.

2.5 Design and Implementation Constraints

- The project should be simple enough to demonstrate in an academic environment.
- The system must use the technologies selected by the project team.
- The website should be responsive for different screen sizes.
- Database operations must follow a consistent relational design.
- Only authorized users should access administrative functions.
- The system depends on the availability of the local development/server environment.

2.6 Assumptions and Dependencies

- Users have access to an internet connection and a supported browser.
- Users provide valid information when making a booking.
- Administrators enter accurate destination and package information.
- The MySQL server is available when data needs to be stored or retrieved.

- External payment, hotel or transportation services may be added later and are not required for the initial academic version.

3. Specific Requirements

3.1 Functional Requirements

ID	Requirement	Description
FR-01	Home Page	The system shall display a home page containing the website name, navigation menu, tourism
FR-02	Destination Listing	The system shall display available destinations with basic information and images.
FR-03	Destination Details	The system shall allow users to view detailed information about a selected destination, includin
FR-04	Package Listing	The system shall display available tour packages with package name, destination, duration, pri
FR-05	Package Details	The system shall allow users to view complete details of a selected package.
FR-06	Search and Filter	The system shall allow users to search or filter tourism content by destination or package inform
FR-07	Registration	The system shall allow a user to register an account when authentication is included.
FR-08	Login	The system shall authenticate registered users before providing protected account features.
FR-09	Booking	The system shall allow users to submit a booking request by entering required travel and perso
FR-10	Booking Validation	The system shall validate required booking fields and reject incomplete or invalid submissions.
FR-11	Booking Storage	The system shall store valid booking information in the MySQL database.
FR-12	Booking Status	The system shall allow authorized staff to update and review booking status.
FR-13	Contact	The system shall allow users to submit questions or feedback using a contact form.
FR-14	Admin Login	The system shall provide secure access to administrative functions.
FR-15	Destination Management	The administrator shall be able to add, edit, view and delete destination records.
FR-16	Package Management	The administrator shall be able to add, edit, view and delete tour package records.
FR-17	Booking Management	The administrator shall be able to view and manage submitted bookings.
FR-18	User Management	The administrator shall be able to view and manage user records when user accounts are enab
FR-19	Message Management	The administrator shall be able to view contact messages submitted through the website.
FR-20	Database Integration	The backend shall communicate with MySQL to create, read, update and delete required recor

3.2 Booking Workflow

- 1. User opens the Tour Packages or Booking section.
- 2. User selects a destination or tour package.
- 3. User enters name, email, travel date, number of travelers and other required details.
- 4. The frontend validates the entered data.
- 5. The backend receives the request through an Express.js API.
- 6. The backend validates the request again and applies business rules.
- 7. Valid data is stored in MySQL.
- 8. The system returns a success response or booking confirmation.

- 9. The administrator can review and update the booking status.

4. External Interface Requirements

4.1 User Interface

The website should have a simple, attractive and consistent interface. A navigation bar should provide access to Home, Destinations, Packages, Gallery, Booking, About Us and Contact. Buttons and forms should use clear labels. Images should support tourism discovery without making pages difficult to load.

The interface should be responsive so that users can use the website on mobile phones, tablets and desktop computers.

4.2 Software Interfaces

- Frontend communicates with the backend through HTTP requests and API endpoints.
- Backend uses Express.js to handle routes, requests, validation and responses.
- Backend connects to MySQL using a suitable Node.js MySQL driver/library.
- The database stores persistent users, destinations, packages, bookings and messages.

4.3 Communication Interfaces

The system will use HTTP/HTTPS for browser-server communication. In a deployed version, HTTPS should be used to protect login and booking information. API requests should return appropriate status codes and structured responses.

5. Database Requirements

5.1 Database Overview

MySQL will be used as the relational database. The database should avoid unnecessary duplication and should use primary keys and foreign keys where appropriate. Each table should have a clear purpose and relationship with other tables.

5.2 Suggested Tables

Table	Main Fields	Purpose
Users	user_id, name, email, password, role	Stores user and admin account information.
Destinations	destination_id, name, location, description, image	Stores Nepal destination information.
Packages	package_id, destination_id, name, duration, price, description	Stores tour package information.
Bookings	booking_id, user_id, package_id, travel_date, people, status	Stores customer booking requests.
Messages	message_id, name, email, subject, message, created_at	Stores contact and feedback messages.

5.3 Database Relationships

- One destination can have many tour packages.
- One user can create many bookings.

- One package can be selected in many bookings.
- Bookings connect users and packages through foreign keys.
- Messages are independent records associated with website visitors.

6. System Models

6.1 Context Model

The context model represents Explore Nepal as one central system and shows the external people or systems that interact with it. The main external entities are Customer/User, Admin, Tourism Information Provider, Hotel and Accommodation, Transportation Provider, and Payment Gateway/Bank.

- **Customer/User** → **Explore Nepal:** sends searches, user information and booking details.
- **Explore Nepal** → **Customer/User:** provides destinations, packages, booking results and travel information.
- **Admin** → **Explore Nepal:** sends updates to destinations, packages, users and bookings.
- **Explore Nepal** → **Admin:** provides booking records, user information and messages.
- **Tourism Information Provider** → **Explore Nepal:** provides tourism-related information.
- **Hotel/Accommodation** → **Explore Nepal:** may provide accommodation availability and related information.
- **Transportation Provider** → **Explore Nepal:** may provide transport options and availability.
- **Payment Gateway/Bank** ↔ **Explore Nepal:** may process payment information in a future enhanced version.

6.2 Use Case Model

The main actor is the Customer/User. Customer use cases include registering, logging in, viewing destinations, viewing packages, searching, booking a tour and sending messages. The Administrator actor can log in, manage destinations, manage packages, manage users, review bookings and manage contact messages.

6.3 Activity Model – Booking

- Start → Open website → Select package → Enter booking information → Validate information → Submit booking → Store booking in database → Show confirmation → End.
- If validation fails, the system displays an error message and asks the user to correct the information before submission.

7. Non-Functional Requirements

7.1 Usability

- The website shall use clear navigation and readable text.
- Forms shall provide understandable labels and validation messages.
- Important actions such as Book Now and Submit shall be easy to identify.
- The interface shall be responsive on common screen sizes.

7.2 Performance

- Normal pages should load quickly under normal network and server conditions.
- Database queries should be designed efficiently.
- Large images should be optimized before being displayed.

7.3 Reliability

- The system should handle invalid input without crashing.
- The backend should return meaningful error responses.
- Important booking data should be stored consistently.

7.4 Maintainability

- Frontend, backend and database code should be organized into logical components.
- Meaningful names and comments should be used where necessary.
- Configuration such as database credentials should not be hard-coded into public frontend code.

7.5 Portability

The website should work across common modern browsers and should adapt to desktop, tablet and mobile screen sizes.

8. Security Requirements

- Passwords must not be stored as plain text in a production system; password hashing should be used.
- Administrative pages must require authorization.
- User input must be validated on both client and server sides.
- Database queries should use parameterized statements or an ORM/query mechanism that prevents SQL injection.
- Sensitive configuration values should be stored securely using environment variables.
- HTTPS should be used in deployment.
- Sessions or tokens should be protected from unauthorized access.
- The system should restrict administrative actions to authorized roles.

9. Project Constraints, Assumptions and Dependencies

9.1 Constraints

- Limited academic project time and resources.
- The initial version may not include real-time hotel, airline or payment integrations.
- Tourism information may be entered manually by the administrator.
- The system scope should remain manageable for the student group.

9.2 Dependencies

- Node.js and npm must be available for backend development.
- MySQL must be running for database operations.
- Required frontend and backend packages must be installed.
- The website depends on browser and network availability.

10. Future Enhancements

- Online payment gateway integration.
- Google Maps or other map integration.
- Hotel and flight booking integration.
- Real-time availability information.
- User reviews and ratings.
- Wishlist or favorite destinations.
- Email/SMS booking notifications.
- Multilingual support including English, Nepali and Korean.
- AI-based travel recommendations.
- Weather information for selected destinations.
- Mobile application version.

11. Acceptance Criteria

- Users can open the website and navigate between the main pages.
- Users can view destination information.
- Users can view tour package details.
- Users can submit a valid booking request.
- Invalid or incomplete booking information is rejected with a clear message.
- Valid booking information is stored in MySQL.
- Administrators can log in to protected management functions.
- Administrators can manage destination and package records.
- Administrators can view submitted bookings.
- The website is usable on mobile and desktop screens.

12. Conclusion

Explore Nepal – Travel and Tourism Website is a full-stack web application designed to make Nepal tourism information easier to discover and use. The system combines a responsive frontend, a Node.js and Express.js backend, and a MySQL database. The main goal is to provide destination information, tour packages and a simple booking process while giving administrators the ability to manage website content and booking records.

This SRS provides a clear foundation for the Group C development team. By following the requirements in this document, the team can proceed through system design, database design, implementation, testing and final presentation in an organized way. The project also demonstrates important software engineering concepts such as requirements analysis, system modeling, user interaction, database integration, security, and maintainable web development.

Appendix A – Group C Information

No.	Name	Student ID	Role / Suggested Responsibility
1	Bogati Mamata	2401470	Frontend / Documentation / Testing
2	Bhusal Kalpana	2401476	Frontend / UI Design
3	Tamang Sujina	2401537	Backend / API
4	Bhatta Pabitra	2401467	Database / Testing
5	Mahato Amit Kumar	2401455	Backend / Integration / Documentation

Note: The suggested responsibilities above can be changed according to the actual work distribution agreed by Group C.

Appendix B – Suggested Database Structure

- Users(user_id PK, name, email, password, role)
- Destinations(destination_id PK, name, location, description, image)
- Packages(package_id PK, destination_id FK, name, duration, price, description)
- Bookings(booking_id PK, user_id FK, package_id FK, travel_date, people, status)
- Messages(message_id PK, name, email, subject, message, created_at)